

Practical Part - Question 1

Monday, 18 May 2026 18:19

Important note regarding the practical part: after writing all the code in the sentiment_start.py skeleton provided by the course, we converted all the code to .ipynb format and did all the training and testing on Google Collab in order to have access to their GPU's.
That's why we were able to reach such a high number of training steps.
#####

In this experiment We compared two recurrent architectures for sentiment classification: a simple Elman RNN and a GRU. Both models process the review word by word, where each word embedding is combined with the previous hidden state. After the full review is processed, the final hidden state is passed to an output MLP that predicts the two sentiment logits.

Core RNN cell code:

```
# x shape: [batch_size, input_size]
# hidden_state shape: [batch_size, hidden_size]
combined = torch.cat(tensors=(x, hidden_state), dim=1)
hidden = torch.tanh(self.in2hidden(combined))
```

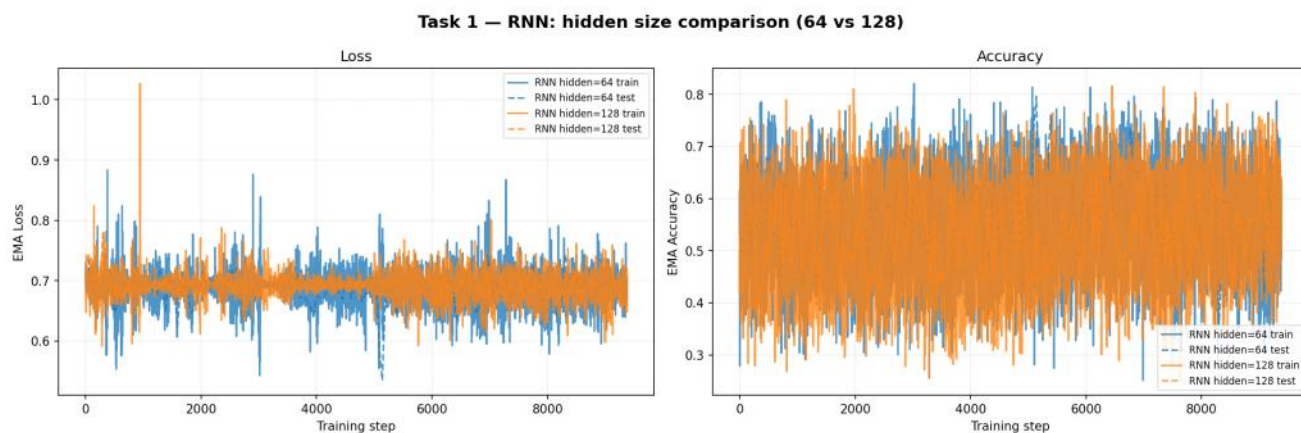
For the RNN, We used a single fully-connected layer that receives the concatenation of the current word embedding and the previous hidden state. We applied tanh to produce the next hidden state, because it is the standard Elman RNN activation and keeps the hidden representation bounded and centered around zero.

Core GRU cell code:

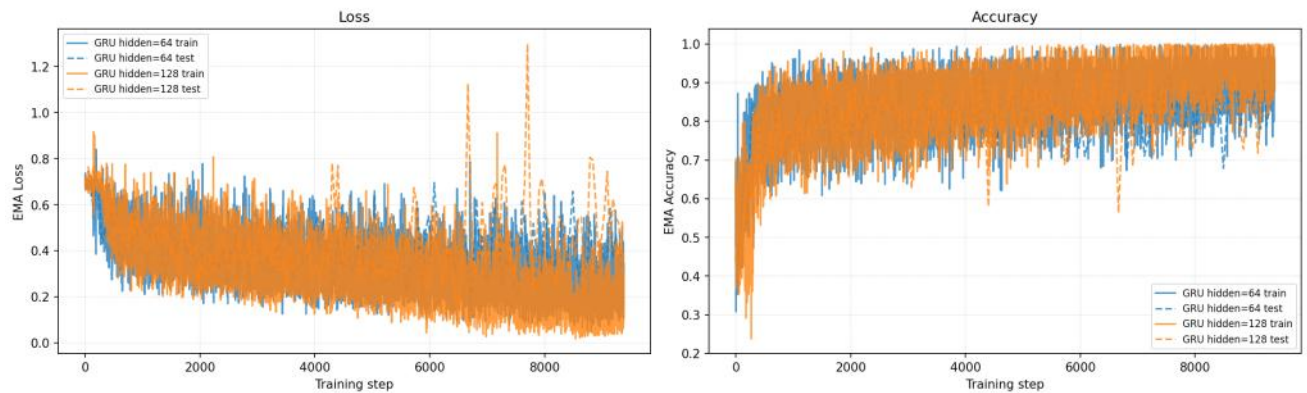
```
# Reset gate and update gate
r = torch.sigmoid(self.reset_gate(combined))
z = torch.sigmoid(self.update_gate(combined))
```

For the GRU, We used learnable reset and update gates, each computed by a single fully-connected layer followed by a sigmoid. These gates allow the model to control how much previous information to keep or reset, which should help compared to the simple RNN when sentiment depends on longer context or late changes in the review.

We have tested both networks with two different hidden layer sizes: 64, 128 and checked which one was better:

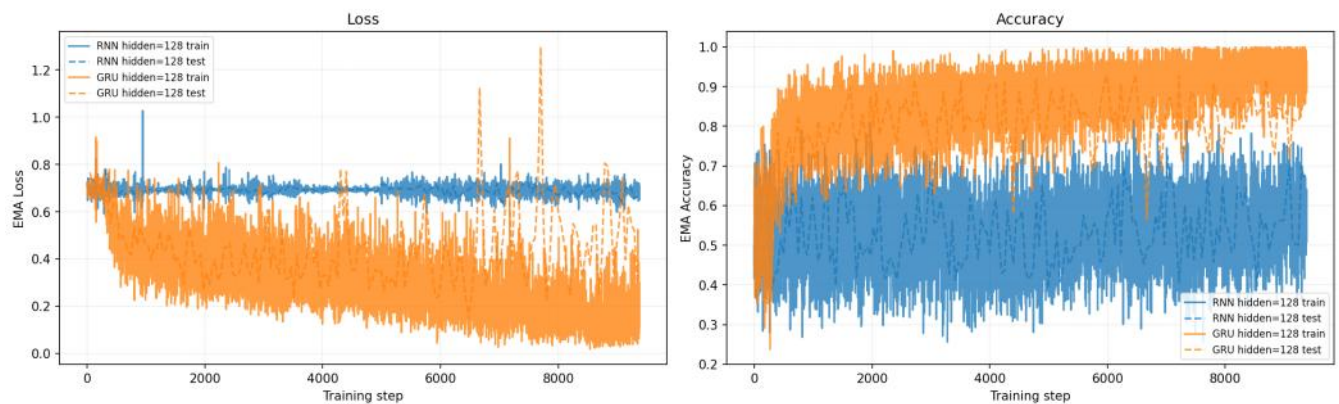


Task 1 — GRU: hidden size comparison (64 vs 128)



We can see that in the GRU hidden layer size 128 was better and for the RNN there was no significant difference. Now the comparison between both models (both hidden size 128):

Task 1 — RNN vs GRU (hidden=128)



The GRU is better suited for this sequence task because its update/reset gates allow it to preserve useful information across the review, while the simple RNN repeatedly overwrites the hidden state and suffers more from vanishing gradients.

Additional test reviews for both models are:

[Positive start + neutral tail — does the model remember early positive signal?]

Review : "i loved this movie the acting was wonderful and the story was moving the rest of the film follows family through several ordinary scenes in ..."

True label: positive

RNN prediction: negative $P(\text{pos})=0.4127$ $P(\text{neg})=0.5873$ WRONG

GRU prediction: positive $P(\text{pos})=0.9984$ $P(\text{neg})=0.0016$ CORRECT

[Positive buildup, negative reversal at the end — does the model update on late signal?]

Review : "the cinematography was beautiful the actors were famous the music was elegant and the scenes looked expensive but the movie was ultimately boring empty and ..."

True label: negative

RNN prediction: negative $P(\text{pos})=0.4127$ $P(\text{neg})=0.5873$ CORRECT

GRU prediction: negative $P(\text{pos})=0.0000$ $P(\text{neg})=1.0000$ CORRECT

The custom reviews show that the GRU handles both long-range sentiment and late reversals better than the simple RNN. The RNN tends to produce low-confidence and nearly constant predictions, suggesting that it struggles to preserve useful information through the 100-word sequence. The GRU's gates allow it to selectively keep or update memory, which explains its stronger performance on these examples.

Practical Part - Question 2

Monday, 18 May 2026 18:19

Applying the global average pooling code:

```
# Creates a mask of shape [batch_size, num_words] (1 for real words, 0 for padding)
mask = (reviews.abs().sum(dim=-1) > 0).float()

# sub_score shape: [batch_size, num_words, 2]
# Expand mask to match sub_score shape: [batch_size, num_words, 1]
expanded_mask = mask.unsqueeze(-1)

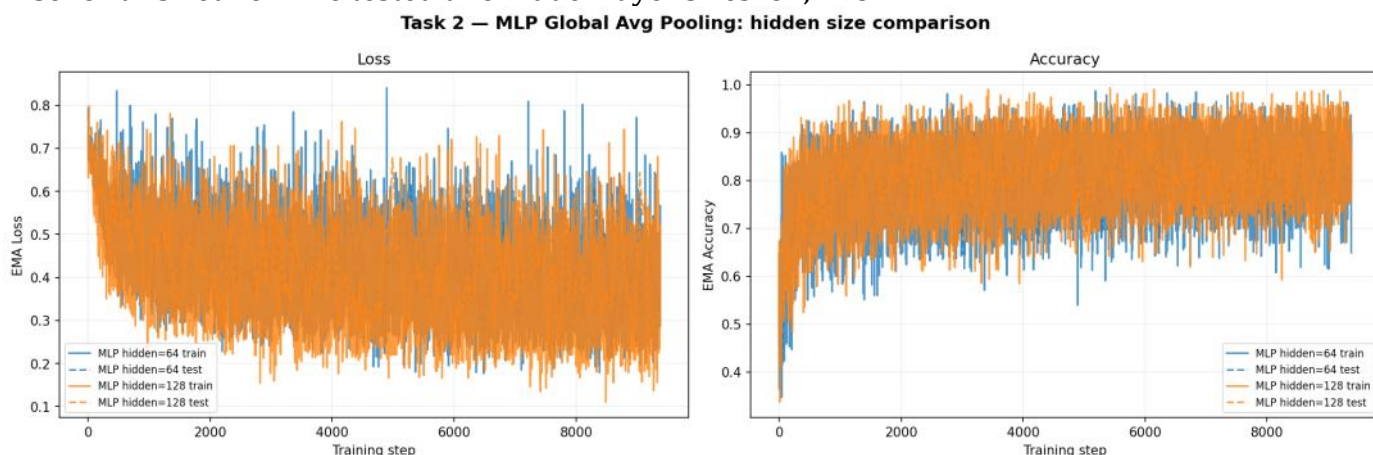
# Zero out padded sub-scores
masked_sub_scores = sub_score * expanded_mask

# Sum the scores and divide by the actual number of words (sum of the mask)
output = masked_sub_scores.sum(dim=1) / mask.sum(dim=1).unsqueeze(-1)
```

The MLP produces a 2-dimensional logit vector for each word, so `sub_score` has shape `[batch_size, num_words, 2]`. Since shorter reviews are padded with zero vectors, I first create a mask that equals 1 for real words and 0 for padding words. Multiplying by this mask removes the contribution of padded words. Then I apply global average pooling by summing the per-word logits over the word dimension and dividing by the number of real words. This gives one final 2-dimensional logit vector per review.

We **average these logits first** in order to aggregate the evidence from all real words into one review-level logit vector. Only after this aggregation should the prediction be interpreted as a probability distribution through softmax. If softmax were applied before averaging, each word would first become an independent probability vote, and strong evidence would be compressed into the range `[0,1]` before being combined. Since the training loss is `CrossEntropyLoss`, which expects raw logits and applies log-softmax internally, averaging logits before the final loss is also the natural implementation.

Also for this network we tested two hidden layer sizes: 64, 128.



Error analysis

The MLP is mostly a word-level sentiment detector. It performs well when the review contains sentiment words whose individual polarity matches the full review label. Its mistakes happen when the global meaning depends on contrast, sarcasm, or reversal. Since each word is scored independently, positive words inside a negative sentence can cause false positives, and negative expectation words inside a positive sentence can cause false negatives.

True Positive:

"this movie was absolutely fantastic masterpiece of cinema with brilliant acting and gripping story that kept me on the edge of my seat the entire time"

Final avg logits: pos=5.5442 neg=-5.2738

Final probs : pos=1.0000 neg=0.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
this	0.361000	-0.385500	0.6784	0.3216
movie	0.255200	-0.208800	0.6140	0.3860
was	-0.245200	0.253700	0.3778	0.6222
absolutely	1.935700	-1.410200	0.9660	0.0340
fantastic	36.274101	-34.286800	1.0000	0.0000
masterpiece	31.911900	-30.271700	1.0000	0.0000
of	0.766400	-0.811900	0.8290	0.1710
cinema	2.761300	-2.563200	0.9952	0.0048
with	0.335400	-0.307900	0.6555	0.3445
brilliant	36.050999	-34.760601	1.0000	0.0000
acting	-3.560700	3.691800	0.0007	0.9993
and	2.977800	-3.005000	0.9975	0.0025
gripping	26.750000	-26.188400	1.0000	0.0000
story	2.115000	-2.023400	0.9843	0.0157
that	0.137200	-0.198000	0.5830	0.4170
kept	2.362400	-2.137400	0.9890	0.0110
me	1.205800	-1.172100	0.9151	0.0849
on	0.826600	-0.750000	0.8287	0.1713
the	0.055100	-0.110400	0.5413	0.4587
edge	0.362000	-0.242200	0.6466	0.3534
of	0.766400	-0.811900	0.8290	0.1710
my	7.286800	-7.113000	1.0000	0.0000
seat	-0.339500	0.428200	0.3170	0.6830
the	0.055100	-0.110400	0.5413	0.4587
entire	-8.586100	8.498600	0.0000	1.0000
time	1.327600	-1.121300	0.9205	0.0795

True Negative:

"this movie was terrible and painfully boring the acting was wooden and the plot made no sense whatsoever"

Final avg logits: pos=-11.9526 neg=12.1047

Final probs : pos=0.0000 neg=1.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
this	0.361000	-0.385500	0.6784	0.3216

movie	0.255200	-0.208800	0.6140	0.3860
was	-0.245200	0.253700	0.3778	0.6222
terrible	-50.133999	50.679199	0.0000	1.0000
and	2.977800	-3.005000	0.9975	0.0025
painfully	-24.653500	25.439199	0.0000	1.0000
boring	-51.409100	51.477600	0.0000	1.0000
the	0.055100	-0.110400	0.5413	0.4587
acting	-3.560700	3.691800	0.0007	0.9993
was	-0.245200	0.253700	0.3778	0.6222
wooden	-51.132599	51.291801	0.0000	1.0000
and	2.977800	-3.005000	0.9975	0.0025
the	0.055100	-0.110400	0.5413	0.4587
plot	-5.375000	5.273500	0.0000	1.0000
made	-3.677700	3.717000	0.0006	0.9994
no	-9.134300	9.203000	0.0000	1.0000
sense	0.450900	-0.202200	0.6577	0.3423
whatsoever	-22.711500	23.630600	0.0000	1.0000

False Positive:

"a masterpiece of wasted potential gripping in its absolute pointlessness with brilliant performers and fantastic score that somehow make this the most surprisingly hollow cinema experience of the year"

Final avg logits: pos=4.0671 neg=-3.6560

Final probs : pos=0.9996 neg=0.0004

word	pos_logit	neg_logit	pos_prob	neg_prob
a	0.849600	-0.851900	0.8457	0.1543
masterpiece	31.911900	-30.271700	1.0000	0.0000
of	0.766400	-0.811900	0.8290	0.1710
wasted	-34.817501	35.127899	0.0000	1.0000
potential	-16.006800	17.110001	0.0000	1.0000
gripping	26.750000	-26.188400	1.0000	0.0000
in	0.977600	-0.853000	0.8618	0.1382
its	5.204200	-4.922300	1.0000	0.0000
absolute	-1.276600	1.073300	0.0871	0.9129
pointlessness	-9.052700	9.248300	0.0000	1.0000
with	0.335400	-0.307900	0.6555	0.3445
brilliant	36.050999	-34.760601	1.0000	0.0000
performers	-0.939900	1.447200	0.0842	0.9158
and	2.977800	-3.005000	0.9975	0.0025
fantastic	36.274101	-34.286800	1.0000	0.0000
score	7.160200	-6.964600	1.0000	0.0000
that	0.137200	-0.198000	0.5830	0.4170

somehow	-15.335900	15.849700	0.0000	1.0000
make	-6.588300	6.665000	0.0000	1.0000
this	0.361000	-0.385500	0.6784	0.3216
the	0.055100	-0.110400	0.5413	0.4587
most	8.296400	-7.755400	1.0000	0.0000
surprisingly	29.936701	-29.091000	1.0000	0.0000
hollow	-8.176700	8.502400	0.0000	1.0000
cinema	2.761300	-2.563200	0.9952	0.0048
experience	15.870900	-14.583200	1.0000	0.0000
of	0.766400	-0.811900	0.8290	0.1710
the	0.055100	-0.110400	0.5413	0.4587
year	2.641700	-2.215100	0.9923	0.0077

False Negative:

"going in expecting something terrible and wooden dreading yet another boringly predictable story was completely blown away this was not bad at all and far exceeded my painfully low expectations"

Final avg logits: pos=-7.7168 neg=7.8681

Final probs : pos=0.0000 neg=1.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
going	-2.788600	2.963000	0.0032	0.9968
in	0.977600	-0.853000	0.8618	0.1382
expecting	6.342800	-6.275300	1.0000	0.0000
something	-6.846900	7.072900	0.0000	1.0000
terrible	-50.133999	50.679199	0.0000	1.0000
and	2.977800	-3.005000	0.9975	0.0025
wooden	-51.132599	51.291801	0.0000	1.0000
dreading	-16.689400	17.493999	0.0000	1.0000
yet	2.286300	-2.246100	0.9894	0.0106
another	-4.629700	4.850200	0.0001	0.9999
boringly	-22.585100	22.997400	0.0000	1.0000
predictable	-26.673300	27.089800	0.0000	1.0000
story	2.115000	-2.023400	0.9843	0.0157
was	-0.245200	0.253700	0.3778	0.6222
completely	-1.584400	1.930600	0.0289	0.9711
blown	-5.335600	5.363700	0.0000	1.0000
away	0.008000	-0.025900	0.5085	0.4915
this	0.361000	-0.385500	0.6784	0.3216
was	-0.245200	0.253700	0.3778	0.6222
not	-3.256800	3.167200	0.0016	0.9984
bad	-27.082600	26.968700	0.0000	1.0000
at	0.043300	-0.093500	0.5341	0.4659

all	0.995300	-1.002100	0.8805	0.1195
and	2.977800	-3.005000	0.9975	0.0025
far	-4.442400	4.358500	0.0002	0.9998
exceeded	0.875100	-1.582200	0.9211	0.0789
my	7.286800	-7.113000	1.0000	0.0000
painfully	-24.653500	25.439199	0.0000	1.0000
low	-11.468600	12.563400	0.0000	1.0000
expectations	1.042700	-1.083700	0.8934	0.1066

Practical Part - Question 3

Monday, 18 May 2026 18:19

ExLRestSelfAtten class code:

We use this class as the 'combined' model where we took the model from task 2 and added the new Self Attention layer to it.

```
class ExLRestSelfAtten(nn.Module): 2 usages new *
    def __init__(self, input_size, output_size, hidden_size): new *
        super(ExLRestSelfAtten, self).__init__()

        self.input_size = input_size
        self.output_size = output_size
        self.hidden_size = hidden_size
        self.atten_size = atten_size
        self.window_size = 2 * self.atten_size + 1
        self.sqr_hidden_size = np.sqrt(float(hidden_size))

        self.ReLU = nn.ReLU()
        self.dropout = nn.Dropout(p=0.2)
        self.softmax = nn.Softmax(dim=2)

        # First token-wise MLP projection: word embedding -> hidden representation
        self.layer1 = MatMul(input_size, hidden_size)

        # Learnable single-head attention matrices
        self.W_q = MatMul(hidden_size, hidden_size, use_bias=False)
        self.W_k = MatMul(hidden_size, hidden_size, use_bias=False)
        self.W_v = MatMul(hidden_size, hidden_size, use_bias=False)

        # Learnable relative positional encoding for offsets:
        # [-5, -4, ..., 0, ..., +4, +5]
        self.pos_embedding = nn.Parameter(
            torch.zeros(1, 1, self.window_size, hidden_size)
        )
        nn.init.normal_(self.pos_embedding, mean=0.0, std=0.02)

        # Continue like the MLP from Task 2, after contextualization
        self.layer2 = MatMul(hidden_size, hidden_size)
        self.layer3 = MatMul(hidden_size, output_size)

    def name(self): 3 usages new *
        return "MLP_atten"
```



```

def forward(self, x): new *
    # Token-wise embedding projection
    x = self.layer1(x)
    x = self.ReLU(x)
    x = self.dropout(x)
    # We build local neighborhoods of size 2*atten_size + 1 around each word.
    padded = pad(x, (0, 0, self.atten_size, self.atten_size, 0, 0))
    x_nei = []
    for k in range(-self.atten_size, self.atten_size + 1):
        x_nei.append(torch.roll(padded, k, dims=1))
    x_nei = torch.stack(x_nei, dim=2)
    # Remove the padded positions from the center-word axis
    # final x_nei shape: [batch_size, num_words, window_size, hidden_size]
    x_nei = x_nei[:, self.atten_size:-self.atten_size, :, :]
    # Add relative positional encoding to the neighbor vectors.
    # This tells the model whether a neighbor is left/right/near/far.
    x_nei = x_nei + self.pos_embedding
    # Single-head restricted self-attention
    query = self.W_q(x)          # [batch_size, num_words, hidden_size]
    keys = self.W_k(x_nei)       # [batch_size, num_words, window_size, hidden_size]
    vals = self.W_v(x_nei)       # [batch_size, num_words, window_size, hidden_size]
    # Attention scores: dot product between each word query
    # and the keys of its local neighbors.
    atten_scores = (query.unsqueeze(2) * keys).sum(dim=-1)
    atten_scores = atten_scores / self.sqrt_hidden_size
    # Attention weights over the local window
    atten_weights = self.softmax(atten_scores)
    # Weighted average of local value vectors
    context = (atten_weights.unsqueeze(-1) * vals).sum(dim=2)
    # Residual connection: keep original word info + local contextual info
    x = x + context
    # Token-wise classifier, same idea as Task 2
    x = self.layer2(x)
    x = self.ReLU(x)
    x = self.dropout(x)
    x = self.layer3(x)
    # These are per-word contextualized logits.
    return x, atten_weights

```

The highlighted sections shows where the self attention is taking place in this new class. The second one is where we compute the attention scores using query and keys.

The query is computed from the current word representation, while the keys and values are computed from the local neighborhood of 11 words: 5 before, the current word, and 5 after. The attention score is the scaled dot product between the current word's query and each local key. Softmax is applied across the local-window dimension, so each word receives attention weights over its nearby words.

Practical Part - Question 4

Monday, 18 May 2026 18:19

After applying the self attention layer to the model from task 2 the main thing we can conclude is that attention changed the local sub-scores, but the final average prediction can still be dominated by many strong sentiment words. The self-attention layer gives each word local context, but the model is still limited by the final average-pooling decision rule.

Showing the model performance with the self attention layers on the same examples from task 2:

True Positive:

"this movie was absolutely fantastic masterpiece of cinema with brilliant acting and gripping story that kept me on the edge of my seat the entire time"

Final avg logits: pos=7.5634 neg=-7.0855

Final probs : pos=1.0000 neg=0.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
this	18.613001	-18.486601	1.0000	0.0000
movie	21.159700	-19.690901	1.0000	0.0000
was	6.096100	-6.358400	1.0000	0.0000
absolutely	19.306000	-19.448799	1.0000	0.0000
fantastic	15.882900	-16.158100	1.0000	0.0000
masterpiece	24.211500	-23.422701	1.0000	0.0000
of	-0.062200	0.047900	0.4725	0.5275
cinema	14.204800	-13.464000	1.0000	0.0000
with	1.094400	-0.965900	0.8870	0.1130
brilliant	13.461200	-13.663200	1.0000	0.0000
acting	18.280199	-17.936399	1.0000	0.0000
and	9.098700	-8.718000	1.0000	0.0000
gripping	25.025400	-24.676500	1.0000	0.0000
story	2.399300	-1.700700	0.9837	0.0163
that	-0.292700	0.233000	0.3715	0.6285
kept	-0.054300	0.041700	0.4760	0.5240
me	1.307500	-0.585800	0.8691	0.1309
on	0.869600	-0.061800	0.7174	0.2826
the	0.342700	0.038100	0.5756	0.4244
edge	-0.055200	0.050300	0.4737	0.5263
of	0.540200	0.044100	0.6215	0.3785

my	0.067700	-0.036400	0.5260	0.4740
seat	1.149300	-0.129900	0.7823	0.2177
the	0.100900	0.017100	0.5209	0.4791
entire	2.171800	1.558700	0.6486	0.3514
time	1.728900	-0.750100	0.9227	0.0773

True Negative:

"this movie was terrible and painfully boring the acting was wooden and the plot made no sense whatsoever"

Final avg logits: pos=-10.5252 neg=17.6823

Final probs : pos=0.0000 neg=1.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
this	-20.013800	34.440601	0.0000	1.0000
movie	-21.799101	38.369099	0.0000	1.0000
was	-7.350100	13.253200	0.0000	1.0000
terrible	-17.969200	32.070702	0.0000	1.0000
and	-2.038200	3.681200	0.0033	0.9967
painfully	-22.388100	37.100101	0.0000	1.0000
boring	-22.906099	38.787800	0.0000	1.0000
the	0.084500	0.013600	0.5177	0.4823
acting	-21.516600	32.968399	0.0000	1.0000
was	-15.885000	24.943300	0.0000	1.0000
wooden	-16.518700	24.837700	0.0000	1.0000
and	-2.944400	5.486000	0.0002	0.9998
the	-0.421300	0.351500	0.3159	0.6841
plot	-7.481400	13.796000	0.0000	1.0000
made	-2.394900	4.347600	0.0012	0.9988
no	-1.373300	2.153100	0.0286	0.9714
sense	-4.753900	7.188500	0.0000	1.0000
whatsoever	-1.784100	4.492500	0.0019	0.9981

False Positive:

"a masterpiece of wasted potential gripping in its absolute pointlessness with brilliant performers and fantastic score that somehow make this the most surprisingly hollow cinema experience of the year"

Final avg logits: pos=1.9672 neg=-0.8057

Final probs : pos=0.9412 neg=0.0588

word	pos_logit	neg_logit	pos_prob	neg_prob
a	6.3283	-5.9196	1.0000	0.0000
masterpiece	-7.6843	12.8518	0.0000	1.0000
of	0.0888	-0.0242	0.5282	0.4718
wasted	-9.6678	15.5192	0.0000	1.0000
potential	-4.1556	6.0945	0.0000	1.0000
gripping	-3.8663	5.8892	0.0001	0.9999
in	0.0977	0.0665	0.5078	0.4922
its	0.4413	1.0898	0.3433	0.6567
absolute	4.0699	-4.4249	0.9998	0.0002
pointlessness	7.2780	-7.3065	1.0000	0.0000
with	1.4682	-1.5913	0.9552	0.0448
brilliant	13.4875	-13.7030	1.0000	0.0000
performers	-0.3346	1.4666	0.1417	0.8583
and	4.6152	-4.0965	0.9998	0.0002
fantastic	12.2655	-12.8311	1.0000	0.0000
score	-2.5574	4.1452	0.0012	0.9988
that	0.0370	-0.2219	0.5644	0.4356
somehow	-5.7332	9.3890	0.0000	1.0000
make	-1.8127	3.3103	0.0059	0.9941
this	12.0399	-11.3810	1.0000	0.0000
the	-0.0150	0.0109	0.4935	0.5065
most	8.8828	-8.3261	1.0000	0.0000
surprisingly	13.6525	-13.4936	1.0000	0.0000
hollow	7.9941	-7.7467	1.0000	0.0000
cinema	-8.3882	15.2018	0.0000	1.0000
experience	6.4393	-6.0519	1.0000	0.0000
of	0.4066	-0.2287	0.6537	0.3463
the	0.0036	-0.0036	0.5018	0.4982
year	1.6679	-1.0481	0.9380	0.0620

False Negative:

"going in expecting something terrible and wooden dreading yet another boringly predictable story was completely blown away this was not bad at all and far exceeded my painfully low expectations"

Final avg logits: pos=-5.4960 neg=9.7856

Final probs : pos=0.0000 neg=1.0000

word	pos_logit	neg_logit	pos_prob	neg_prob
going	-4.215100	7.895000	0.0000	1.0000
in	0.248600	0.041800	0.5515	0.4485
expecting	-0.206800	0.481900	0.3343	0.6657
something	-10.056500	15.643500	0.0000	1.0000
terrible	-15.636200	23.680700	0.0000	1.0000
and	-7.047500	11.336700	0.0000	1.0000
wooden	-15.851900	23.806801	0.0000	1.0000
dreading	-13.752300	20.587500	0.0000	1.0000
yet	-6.403700	11.047700	0.0000	1.0000
another	-10.427100	17.746901	0.0000	1.0000
boringly	-12.237800	23.996901	0.0000	1.0000
predictable	-18.742701	32.051899	0.0000	1.0000
story	-11.446100	18.610399	0.0000	1.0000
was	-0.010100	1.296800	0.2130	0.7870
completely	-2.461600	6.848800	0.0001	0.9999
blown	-1.286600	5.781200	0.0009	0.9991
away	0.554600	-0.036900	0.6437	0.3563
this	-5.650700	11.900700	0.0000	1.0000
was	-2.913900	8.305200	0.0000	1.0000
not	0.385400	-0.028000	0.6019	0.3981
bad	-5.000100	10.949500	0.0000	1.0000
at	0.911600	-0.066300	0.7267	0.2733
all	-0.462900	0.744600	0.2301	0.7699
and	2.448200	-2.280400	0.9912	0.0088
far	-0.868200	1.518400	0.0842	0.9158
exceeded	-3.081200	7.008700	0.0000	1.0000
my	-8.641000	13.643900	0.0000	1.0000
painfully	-4.680400	7.781300	0.0000	1.0000
low	-5.473600	8.555100	0.0000	1.0000
expectations	-2.874300	4.716400	0.0005	0.9995

Adding another example of a review where the model from task 2 failed but this one with attention got right:

Review: "the film was not boring at all it was surprisingly fun and engaging from

start to finish"

True label: positive

MLP (task 2) Results:

Predicted label : negative

Final avg logits: pos=-0.1639 neg=0.3029

Final probs: pos=0.3854 neg=0.6146

MLP + Attention (task 4) Results:

Predicted label : positive

Final avg logits: pos=2.2319 neg=-0.4930

Final probs: pos=0.9385 neg=0.0615

Theoretical Part - Question 1

Friday, 15 May 2026 19:11

These are our architecture choices for each scenario:

1. Speech to Text - seq2seq RNN. The data is clearly sequential and RNN addresses that. The seq2seq structure is since we get a long wav file which represents a few sentences, and the output is the actual predicted words, so also many values
2. Answer questions - seq2seq Transformer. Here we clearly need semantic and context understanding, which would be optimal using a transformer. Again here we have a sentence as an input and as an output, which makes seq2seq the needed architecture.
3. Sentiment analysis - Many-to-One RNN. Although context here is important, most of the information is carried by semantic representation of the words in the sentence, and Transformer could be an overkill for this. The input would be a sentence, therefore a "many" input, and the output is usually binary or a normalized number (scale 0-1, 1-10, etc), so a single output parameter should be enough.
4. Image Classification - 1:1 CNN. The CNN itself is needed for getting the visual content of the data in a way an MLP layer is not able to get (that is, by searching for patterns of increasing sizes). We'd eventually get a representation of the image in a latent dimension, and could add a few dense layer on top for the classification process. The output would be a single vector in the size of the amount of classes.
5. Single Word translation - MLP. since context here is irrelevant, and there's no series or internal structure, this is essentially a mapping of two embedding spaces that represent the target and source languages. An MLP could potentially learn this mapping accurately.

Theoretical Part - Question 2

Friday, 15 May 2026 19:23

1. For Text to Image model, we suggest the following architecture:
 - a. Transformer block - the transformer receives the sentence, and extracts using self-attention and the MLP layers at the end, to a latent dimension that maps the sentence intent.
 - b. GAN block - a dual generator-discriminator models that could correctly learn together the visual representation corresponding to the intent representation. We believe this could work based on the assumption that the encoding and decoding of the images could happen in a low-dim latent space.
2. To enable the network to address the region during the attention process, we should add positional encoding. This encoding indicated the relative location of the given input. Using the positional encoding, the network could learn to connect the text tags and regions they represent. Since attention is permutation-invariant, without the positional encoding, the network would not be able to distinguish between data coming from different regions.

Theoretical Part - Question 3

Friday, 15 May 2026 19:45

1. Using the standard spatial output formula:

$$S_n = \text{floor}\left(\frac{S_{n-1} - K + 2P}{\text{Stride}}\right) + 1$$

$$S_0 = 128$$

$$S_1 = \text{floor}\left(\frac{128 - 3 + 2 * 0}{2}\right) + 1 = 63$$

$$S_2 = \text{floor}\left(\frac{63 - 5 + 2 * 2}{1}\right) + 1 = 63$$

$$S_3 = \text{floor}\left(\frac{63 - 3 + 2}{1}\right) + 1 = 63$$

$$S_4 = \text{floor}\left(\frac{63 - 5 + 2 * 0}{2}\right) + 1 = 30$$

So the output map would be of size 30x30

2. Using the formula for reception field:

$$J_n = J_{n-1} * \text{Stride}_n, R_n = R_{n-1} + (K_n - 1) * J_{n-1},$$

With $R_0 = 1, J_0 = 1$

So:

$$J_1 = 1 * 2 = 2, \quad R_1 = 1 + (3 - 1) * 1 = 3$$

$$J_2 = 2 * 1 = 2, \quad R_2 = 3 + (5 - 1) * 2 = 11$$

$$J_3 = 2 * 1 = 2, \quad R_3 = 11 + (3 - 1) * 2 = 15$$

$$J_4 = 2 * 2 = 4, \quad R_4 = 15 + (5 - 1) * 2 = 23$$

So the receptive field size is 23x23.

Theoretical Part - Question 4

Friday, 15 May 2026 20:24

First, we note that if a single Attention + MLP layer is permutation-invariant, then stacking together several such layers would keep this property.

The MLP layer, since it's fully connected, is applied independently to all tokens in the same way. This causes swaps between two input tokens to cause swaps in the output tokens with the same computation in between.

For the attention layer, the situation is similar. The attention process maps all inputs to all queries, and selects the values which correspond to answering that query. When we apply self-attention, the query and the values (and the keys) are all the same, meaning we essentially check the relevance of a token against all other tokens. This makes self-attention permutation invariant as well, because the order does not change the fact that all tokens are queried in pairwise relation.

Positional encoding prevents the self-attention process from being fully permutation invariant, since we map all input to its relative position. This allows the MLP layers to later learn based on that respective position, and assign different weights based on the order of the input token, making it non-permutation invariant.